CHITTAGONG UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Department of Electronics and Telecommunication Engineering

PROJECT REPORT

Design and Implementation of a SAP-1 Architecture with Control Sequencer Using Logisim Evolution

Course No: ETE 404

Course Title: VLSI Technology Sessional

Submitted By	Submitted To
Name: Oesmita Chakma Moon	Arif Istiaque
Student ID: 2008061	Lecturer
	Department of ETE, CUET

Contents

1 Introduction **2**

2 Objectives **2**

3 Key System Features **3**

4 Architectural Design and Component Analysis **4**

4.1 System Architecture Overview 4

4.2 Register Implementation (A and B) 6

4.3 Program Counter (PC) with loader 6

4.4 Memory System and Address Register (MAR) 7

4.5 Instruction Register and Opcode Decoder 9

4.6 Arithmetic Logic Unit (ALU) 10

4.7 Timing Control Generator 10

4.8 Automatic Operation Control Logic 12

4.9 Manual/Loader Operation Control 13

5 Instruction Set Architecture **15**

5.1 Instruction Encoding Scheme 15

6 Assembler **16**

7 Operation **17**

7.1 Fetch–Decode–Execute Cycle 17

8 Running the CPU in Automatic Mode (JUMP + ROTATE Program) **18**

9 Conclusion **23**

1 Introduction

This report documents a complete, reproducible build of a SAP-1 compatible microprocessor implemented in **Logisim-Evolution**, with a hardwired **Control Sequencer**, a **RAM Auto-Loader**, and a **Python assembler**. The design follows a single-bus organization with strict single-driver discipline and a 4-bit address space (16 locations). The unique feature emphasized in this build is an **unconditional JUMP** instruction and **arithmetic-on-store** semantics: `STA` writes $A + B$ to memory and `STS` writes $A - B$ to memory, enabling in-place memory results without an explicit `ADD/SUB` opcode.

The enhanced instruction set comprises `LDA`, `LDB`, `ROL A`, `ROR A`, `ROL B`, `ROR B`, and `HLT`, enabling arithmetic, rotation, and control flow operations. Instruction processing is organized through a systematic fetch–decode–execute sequence, coordinated by a six-phase ring counter and an opcode decoder.

All key modules were designed and simulated to validate timing accuracy and logical functionality. This design serves as an effective educational model for exploring micro-operations, control signal generation, and instruction-level synchronization within a simplified computer architecture.

2 Objectives

1. To design and implement an enhanced 8-bit SAP-1 architecture in Logisim Evolution with added shift and branch instructions for educational analysis.

2. To establish a single-bus system with an 8-bit data path and 4-bit address space, governed by hardwired control for precise timing.

3. To incorporate two operating modes:

   - **Automatic Mode:** Executes the fetch–decode–execute cycle using a six-phase ring counter.
   - **Manual/Loader Mode:** Enables secure ROM-to-RAM data transfer with diagnostic monitoring.

4. To construct a datapath including dual 8-bit registers, a ripple-carry ALU, Program Counter, MAR, SRAM, and an Instruction Register divided into opcode and operand fields.

5. To ensure proper synchronization and timing across all modules through the control sequencer.

6. To validate system performance through instruction execution and verification of correct data transfer and control signal behavior.

3 Key System Features

1. **Enhanced SAP-1 foundation:** The processor builds upon the classical SAP-1 framework and extends it with additional functional modules. It utilizes a single 8-bit data bus and a 4-bit address space (16 memory locations), following a fully hardwired control approach without microprogramming. This design preserves architectural simplicity while clearly illustrating data flow and instruction execution principles.

2. **Expanded instruction set:** The architecture supports nine core instructions: `LDA`, `LDB`, `STA`, `STS`, `ROL A`, `ROR A`, `ROL B`, `ROR B`, `JUMP`, and `HLT`. Together, these enable arithmetic, logical shift, and branching operations, extending the computational capability of the traditional SAP-1 model.

3. **Dual operational modes:** The system can function in two distinct modes:

   - **Automatic Mode:** Executes programs sequentially through a structured fetch–decode–execute cycle driven by a six-phase ring counter (T1–T6).
   - **Manual/Loader Mode:** Allows safe program transfer from ROM or manual input to RAM using handshake-based control signals, supporting debugging and instructional demonstrations.

4. **Hardwired control logic:** Timing pulses from the six-phase ring counter are logically combined with opcode decoder outputs to generate precise micro-operation signals. These signals manage register transfers, ALU operations, memory access, program sequencing, and halt conditions with strict timing synchronization.

5. **Single-driver bus mechanism:** A strict tri-state control policy ensures that only one module drives the system bus at any given time, eliminating data contention and ensuring accurate data transfer between components.

6. **Datapath architecture:** The datapath consists of two 8-bit general-purpose registers (Accumulator A and B), a ripple-carry Arithmetic Logic Unit (ALU) for arithmetic and shift operations, a 4-bit Program Counter (PC) supporting both increment and direct-load modes, a 4-bit Memory Address Register (MAR), a 16×8-bit Static RAM, and an 8-bit Instruction Register (IR) that separates opcode and operand fields.

7. **Opcode decoding:** A 4-to-16 line decoder translates opcode bits into one-hot control signals that directly correspond to each instruction, simplifying signal routing to the control sequencer and minimizing logic complexity.

8. **Deterministic timing control:** Each instruction type is executed over a defined number of timing states. Memory-based instructions such as `LDA` and `LDB`, arithmetic and shift operations (`STA, ROR, ROL`), control instructions (`JUMP, HLT`) all complete within four or fewer cycles, ensuring predictable execution.

9. **Assembler integration and verification:** A dedicated assembler translates human-readable assembly code—using directives like `ORG` and `DATA`—into Logisim-compatible hexadecimal files. This enables convenient program loading and verification through simulation, waveform analysis, and register state observation.

10. **Scalability and pedagogical significance:** The architecture is modular and extensible, allowing additional instructions or status flags (e.g., Zero, Carry) to be easily incorporated. Its visually interpretable design and cycle-by-cycle operation make it an ideal platform for academic laboratories to illustrate processor internals and control signal sequencing.

4 Architectural Design and Component Analysis

4.1 System Architecture Overview

The processor is designed using a single-bus architecture to handle all data transfer operations. Tri-state logic is implemented to guarantee that only one module drives the bus at any given time, while all others remain in a high-impedance state. This shared bus facilitates data exchange among the memory, registers, and ALU, coordinated through a well-defined control sequence. Common bus-driving elements include `pc_out`, `sram_rd`, `address_out_en`, `a_out`, `b_out`, `alu_out`, and `rotater_out`. Meanwhile, bus-receiving components such as `mar_in_en`, `a_in`, `b_in`, and `sram_wr` capture data from the bus when their enable signals are activated.

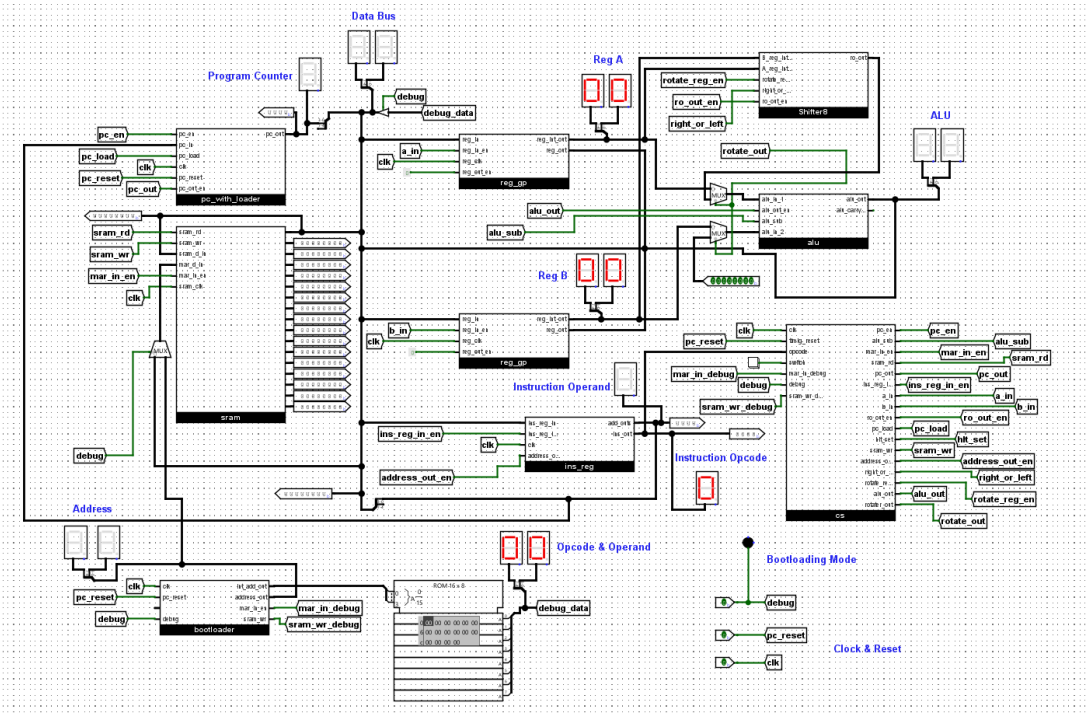

Figure 1: Automatic mode architecture showing single-bus data flow and centralized control.

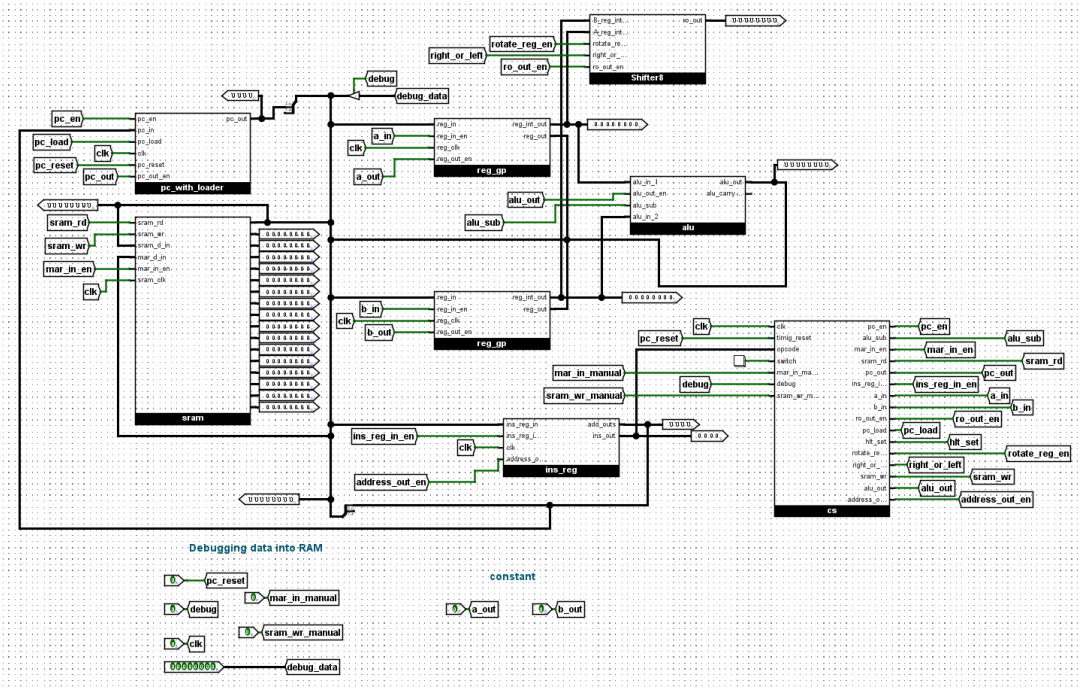

Figure 2: Manual mode architecture illustrating ROM-to-RAM transfer with handshake control.

4.2 Register Implementation (A and B)

The register subsystem provides two 8-bit general-purpose registers—Register A and Register B—each implemented using standard flip-flop-based storage units. Both registers have input and output interfaces controlled by independent enable lines.

- **Purpose:** To store operands and intermediate results during instruction execution.
- **Functionality:** Data from the system bus is latched when `a_in` or `b_in` is asserted. Each register can also drive the bus using its respective `a_out` or `b_out` control. Additionally, internal connections provide continuous data to the ALU for computation without occupying the main bus.

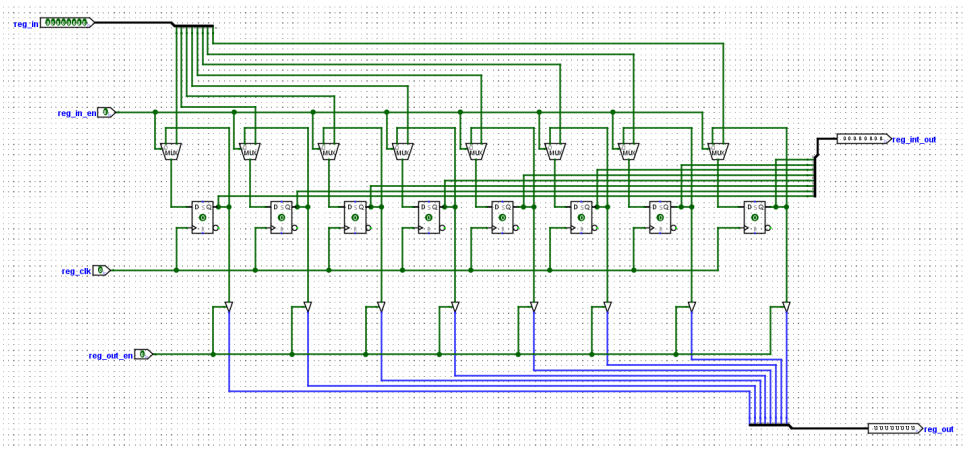

Figure 3: A/B Register subsystem with enable controls and direct ALU linkage.

4.3 Program Counter (PC) with loader

The Program Counter is a 4-bit register capable of incrementing sequentially or loading a new address during branching. It drives the system bus through `pc_out` and accepts new data when `pc_en` is asserted.

- **Purpose:** To hold and manage the address of the next instruction.
- **Operation:**
  - During T1, `pc_out` and `mar_in_en` transfer the PC value to MAR for fetching.
  - During T3, the counter increments (`pc_en`=1).
  - When a `JUMP` instruction is executed, the operand in IR[3:0] replaces the current PC value.

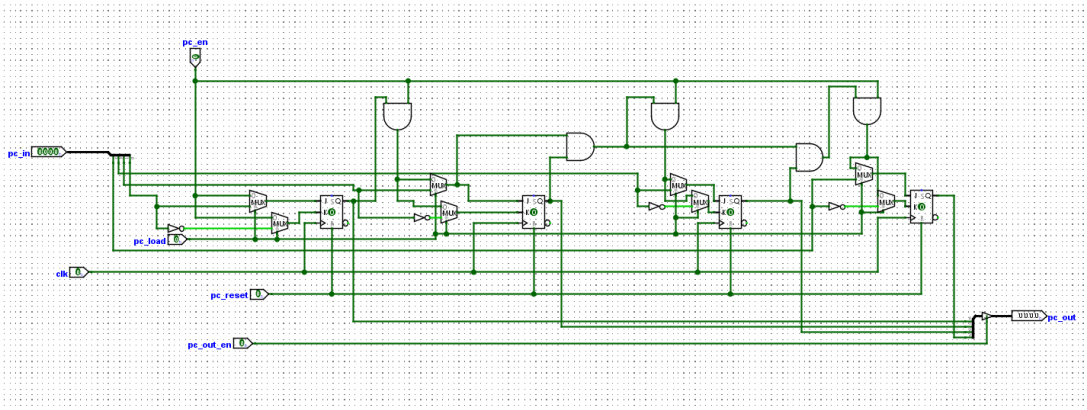

Figure 4: Program Counter supporting sequential increment and jump-based loading.

4.4 Memory System and Address Register (MAR)

The memory subsystem consists of a 4-bit Memory Address Register (MAR) and a 16×8-bit SRAM array. The MAR captures the address of the instruction or data to be accessed, while SRAM handles the storage and retrieval operations.

- **Purpose:** To provide address-driven access to memory.
- **Operation:**
  - During T1, MAR receives the address from PC for instruction fetch.
  - During T4, MAR is loaded from IR[3:0] during operand addressing for instructions such as `LDA`, `LDB`, and `JUMP`.
  - `sram_rd`=1 enables read operations, while `sram_wr`=1 performs writes.

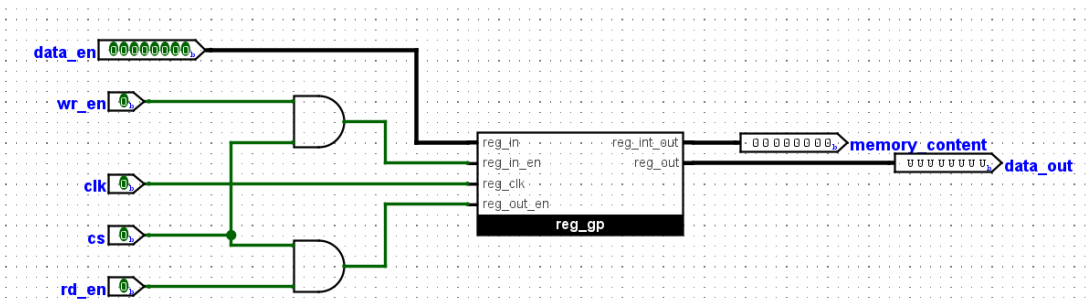

Figure 5: SRAM cell implementation showing D flip-flop-based storage with read, write, and chip-select control signals.

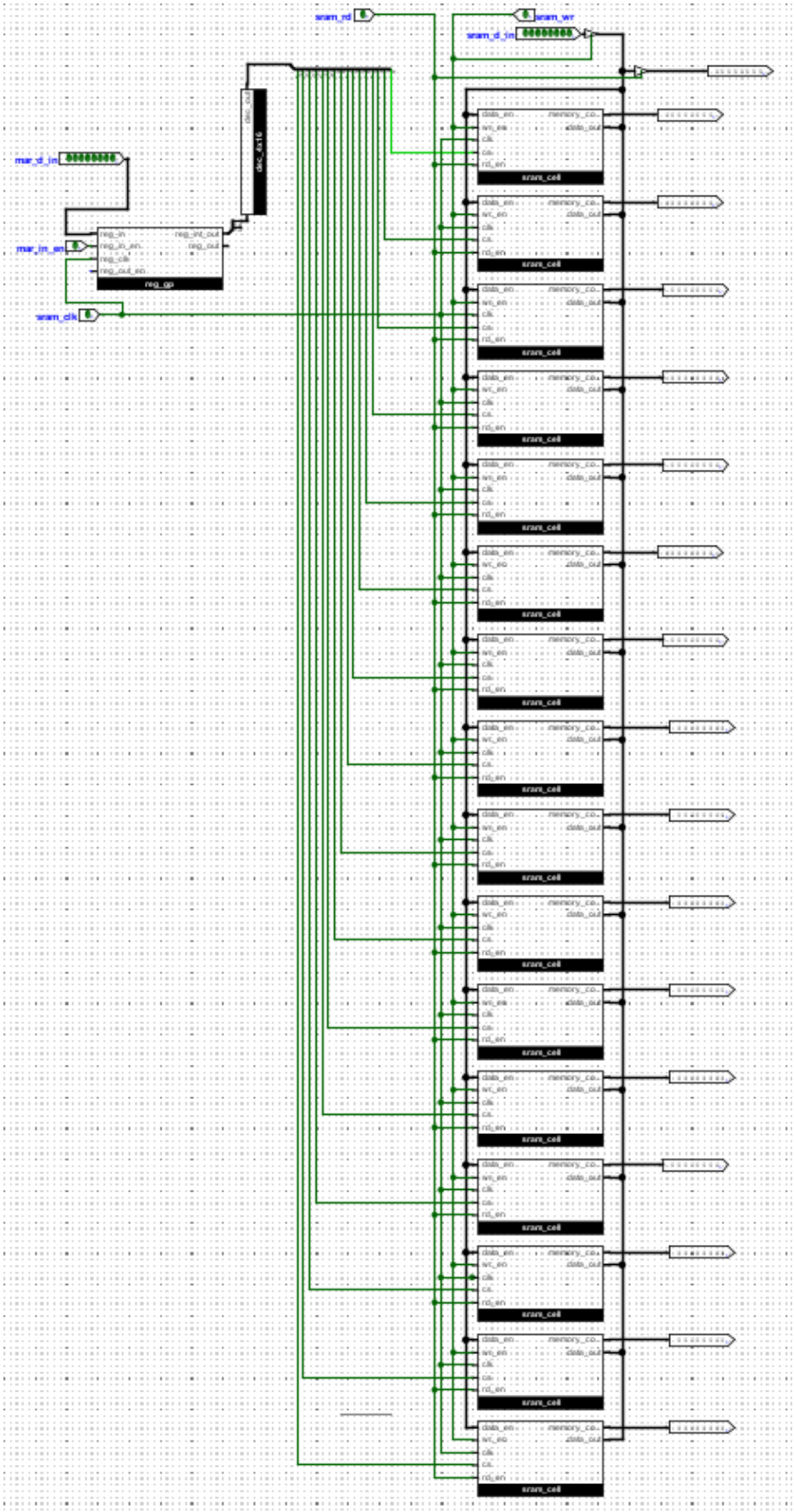

Figure 6: Complete 16×8 SRAM array connected with MAR, supporting address-based data access through read/write control.

4.5 Instruction Register and Opcode Decoder

The Instruction Register (IR) holds the instruction fetched from memory and separates it into two 4-bit fields: opcode and operand. The opcode field is forwarded to the decoder, which produces one-hot control lines for each instruction.

- **Purpose:** To store and decode the current instruction.
- **Operation:**
  - During T2, the instruction is fetched from memory (`sram_rd=1`) and latched into IR (`ins_reg_in_en=1`).
  - IR[7:4] drives the decoder, while IR[3:0] can be placed on the bus for addressing during T4.

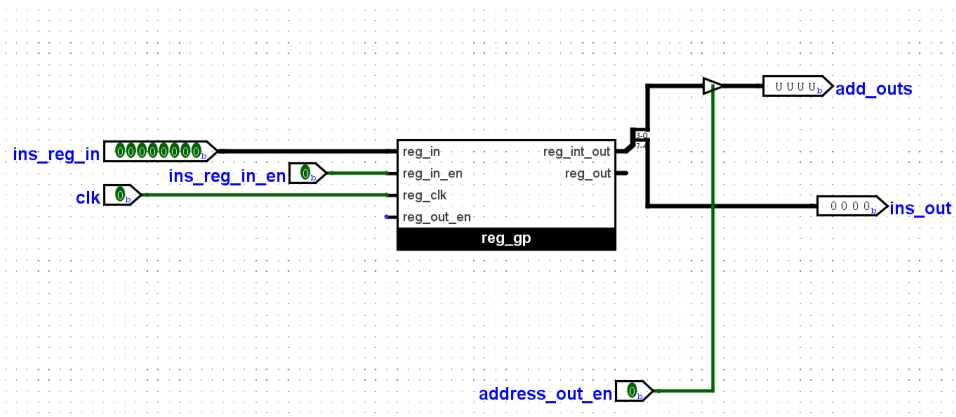

Figure 7: Instruction Register (IR) and Opcode Decoder showing instruction loading, opcode decoding, and operand forwarding paths.

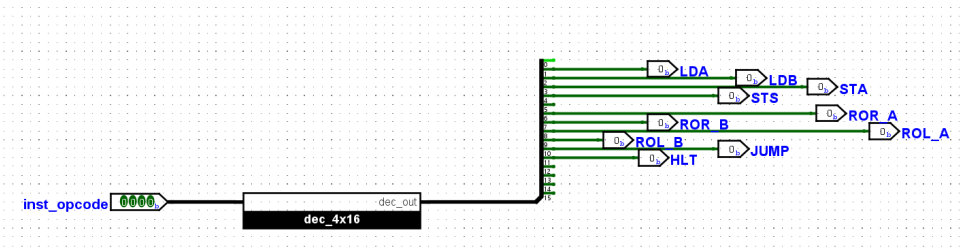

Figure 8: Opcode Decoder converting 4-bit opcodes into one-hot control signals for all supported instructions.

4.6 Arithmetic Logic Unit (ALU)

The ALU performs all arithmetic and logical operations. It supports addition and both left and right shift operations for registers A and B, enhancing computational capability.

- **Purpose:** To execute arithmetic and shift operations based on control signals.
- **Operation:** Inputs are directly taken from internal outputs of Registers A and B, reducing bus traffic. The signal `alu_out`=1 places the computed result onto the bus. Shift operations are controlled using `shift_out` and `right_or_left` signals, determining the direction of bit movement.

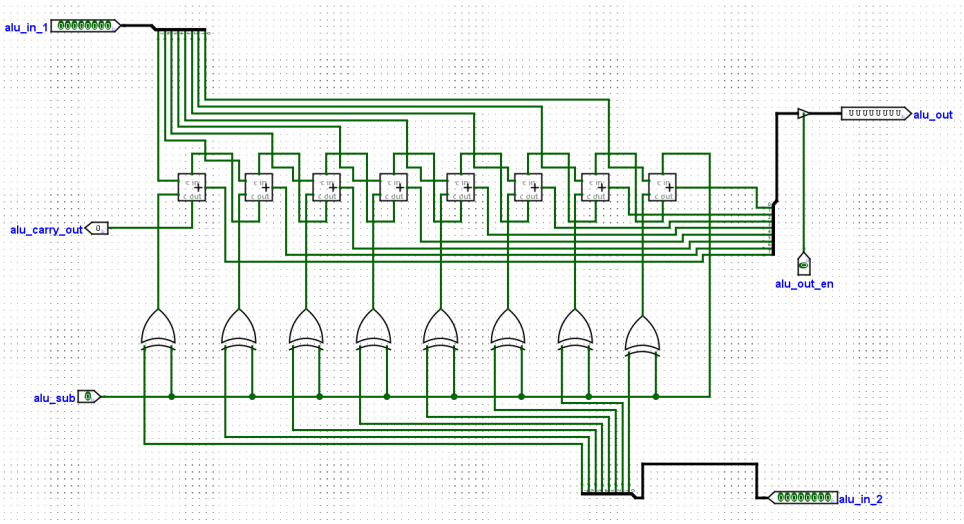

Figure 9: Arithmetic Logic Unit (ALU) implementing 8-bit addition and bidirectional shift operations, utilizing ripple-carry and tri-state bus interfacing.

4.7 Timing Control Generator

The timing control generator employs a 6-phase ring counter that sequentially activates states (T1–T6) for the execution of micro-operations. Each timing phase corresponds to specific control signal activations, ensuring accurate synchronization between fetching, decoding, and executing instructions.

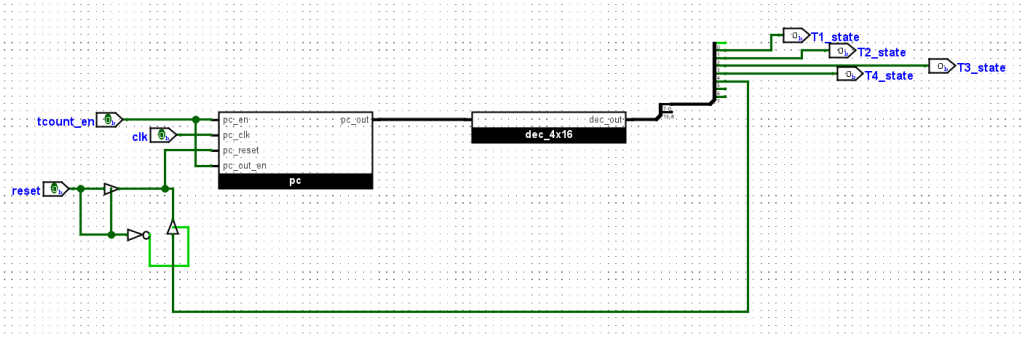

Figure 10: Timing generator using a six-phase ring counter to coordinate fetch–decode–execute cycles for all instruction types.

Universal Fetch Sequence (for all instructions):

- **T1:** `pc_out`, `mar_in_en` → MAR ← PC
- **T2:** `sram_rd`, `ins_reg_in_en` → IR ← M[MAR] and `pc_en` → PC ← PC + 1

Execution Cycles by Instruction:

- **T3:** [common for all] `ins_reg_out_en`, `mar_in_en` → MAR ← IR[3:0];
- **LDA addr:** T4 → `sram_rd`, `a_in` → A ← M[MAR].
- **LDB addr:** T4 → `sram_rd`, `b_in` → B ← M[MAR].
- **STA:** T4 → `alu_out`, `sram_wr_in` → M[MAR] ← ALU
- **ROR A:** T4 → `ro_out_en`, `rotater_out` → A rotated right by one bit.
- **ROL A:** T4 → `ro_out_en`, `rotater_out` , `right_or_left` → A rotated left by one bit.
- **ROR B:** T4 → `ro_out_en`, `rotater_out` , `rotate_reg_en` → B shifted right by one bit.
- **ROL B:** T4 → `ro_out_en`, `rotater_out` , `right_or_left` , `rotate_reg_en` → B rotated left by one bit.
- **JUMP addr:** T4 → `address_out_en`, `pc_load` → PC ← IR[3:0].
- **HLT:** T4 → `hlt_set`=1 → Halts the system by disabling the ring counter.

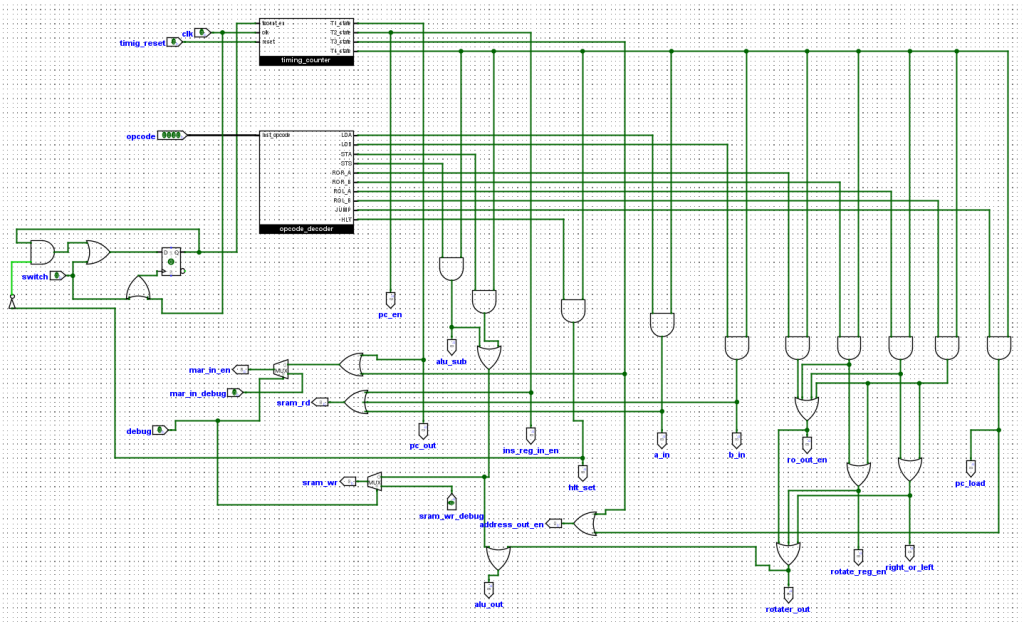

Figure 11: Control Sequencer generating instruction-based pulses synchronized with a six-phase ring counter.

4.8 Automatic Operation Control Logic

The automatic control logic of the SAP-1 processor was developed using Boolean equations to determine the precise timing and sequencing of all micro-operations. The global control enable signal, `c_en`, governs the automatic execution sequence. When `c_en` = 1, the control sequencer activates, allowing the processor to progress through the structured fetch–decode–execute cycle. Conversely, when `c_en` = 0, all control outputs are held inactive, pausing the processor until re-enabled.

A dedicated `debug` input allows the processor to enter manual mode for program loading and diagnostic operations. When `debug` = 1, the automatic sequencer is disabled, enabling direct access to memory and bus signals without interference from the control logic. This separation between `c_en` and `debug` ensures safe mode switching and prevents timing conflicts between manual and automatic operation paths.

The automatic control logic implements a universal fetch sequence applicable to all instructions. During T1, `pc_out` and `mar_in_en` transfer the program counter value to the memory address register (MAR). In T2, `sram_rd` and `ins_reg_in_en` fetch the instruction from memory into the instruction register (IR), while `pc_en` increments the program counter.

Execution cycles vary according to the instruction type. At T3, `ins_reg_out_en` and `mar_in_en` transfer the instruction operand (IR[3:0]) to MAR. During T4, instruction-specific control signals govern operations such as:

- **LDA/LDB:** `sram_rd` with `a_in` or `b_in` to load registers from memory.

- **STA:** `alu_out` and `sram_wr_in` to store data back into memory.
- **Rotation instructions (ROR/ROL A or B):** `ro_out_en`, `rotater_out`, and direction/control signals to shift or rotate register contents.
- **JUMP:** `address_out_en` and `pc_load` to set the program counter from the instruction operand.
- **HLT:** `hlt_set` disables the ring counter, halting all processor activity.

This hardwired design ensures that all register transfers, ALU operations, memory accesses, and rotations occur exclusively within their designated timing windows. By combining decoded instruction signals with precise timing pulses (T1–T4), the system maintains a conflict-free bus environment and deterministic execution, eliminating the need for microcoded sequencing while providing stable operation in both manual and automatic modes.

4.9 Manual/Loader Operation Control

In the **Manual/Loader mode**, the control sequence is modified through the activation of the `debug = 1` signal. When this signal is asserted, the automatic fetch–decode–execute control path is disabled, allowing safe manual program loading into RAM from the ROM module. During this period, the processor remains isolated from automatic control signals, preventing unintended data flow through the bus.

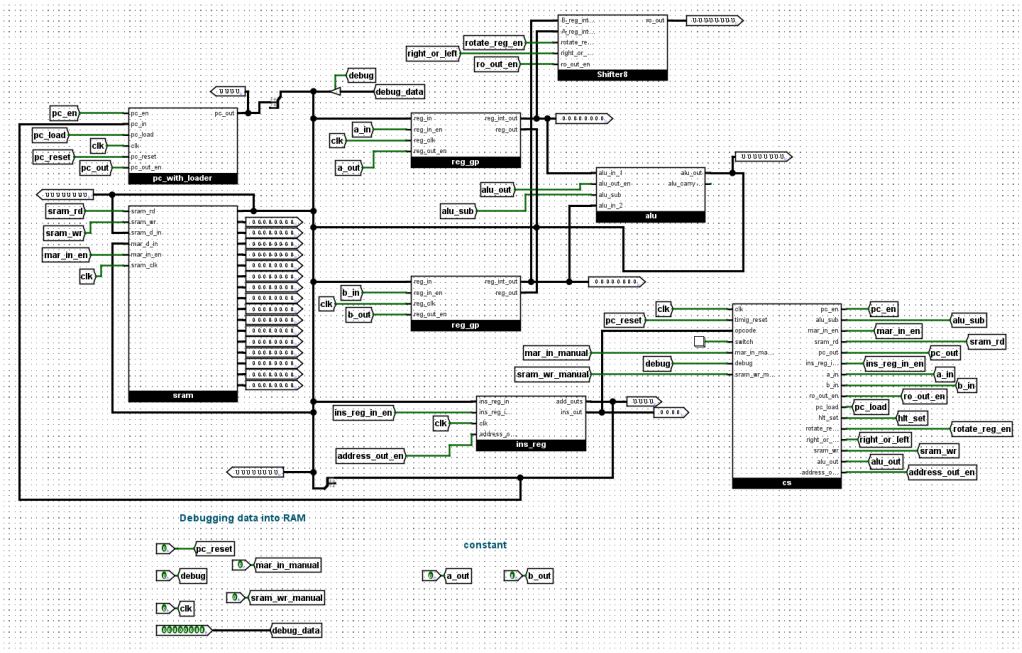

Figure 12: Auto sequencer with debug-enabled manual RAM access.

Bootloader operation and hand–off. When `debug` is asserted (`debug`=1), a small two–phase bootloader takes exclusive control of the data bus and memory interface while the control sequencer is masked (`cs_en`=0). In **Phase–1** (`phase_1`) the circuit drives the address byte `0000:CTR[3:0]` onto the system bus (via an 8–bit tri–state) and asserts `mar_in_en` to capture the address into the internal MAR. In **Phase–2** (`phase_2`) the ROM data is enabled onto the bus and `sram_wr` is asserted to write the byte into SRAM; on this phase the 4–bit address counter `CTR` increments. Thus, every two clock pulses transfer one byte, and the sixteen locations `0x0...0xF` are copied in order without overlap. During this mode, all automatic CU drivers are gated off (every CU output enable is ANDed with `cs_en`), guaranteeing no bus contention.

After the final write, the **hand–off is manual**: the operator deasserts `debug` (`debug`=0) and toggles the CS switch high. At this point the bootloader tri–states are inactive, the control sequencer is re–enabled, and program execution proceeds automatically under the fetch–decode–execute cycle. A brief `pc_reset` to `0000` is recommended before running.

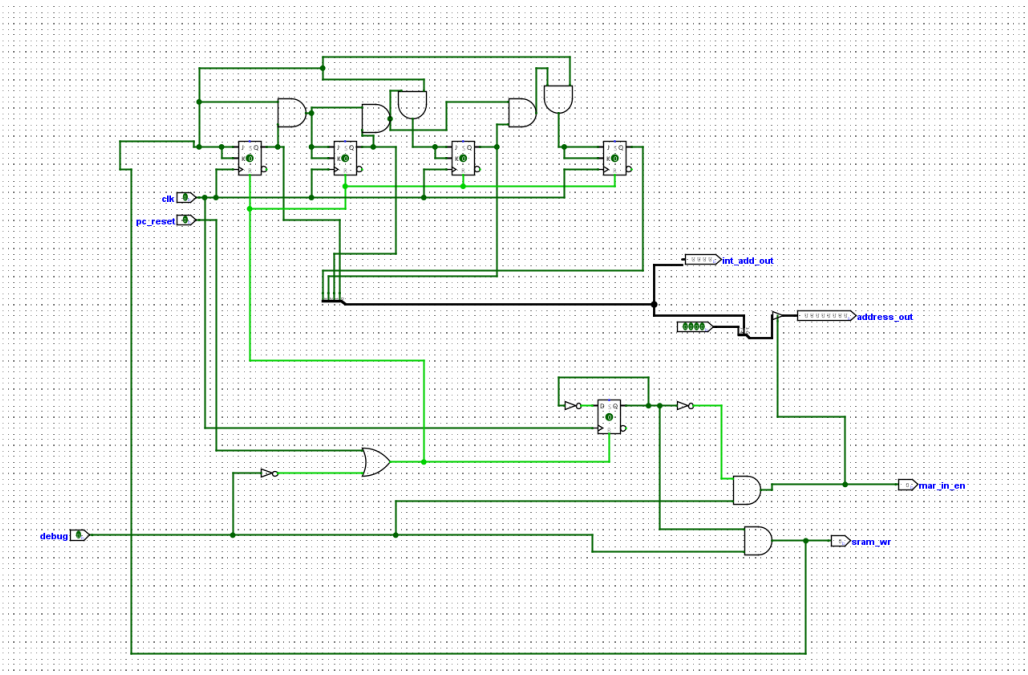

Figure 13: Two-phase ROM→SRAM bootloader with manual hand-off. In `debug` mode, `phase_1` latches the address and `phase_2` writes data and increments the counter. After loading, the operator clears `debug` and asserts `cs_en` to start the automatic sequencer.

This arrangement provides a clean separation of responsibilities: while `debug`=1, the loader has full control and all CU signals remain masked; after the user hand–off, the system resumes normal automatic execution with the control sequencer.

5 Instruction Set Architecture

5.1 Instruction Encoding Scheme

The instruction format utilizes the upper four bits (`IR[7:4]`) to represent the opcode and the lower four bits (`IR[3:0]`) to represent the operand or address field. This allows up to 16 unique instructions, of which ten are utilized in the current design.

Table 1: Instruction Set Summary

Instruction Code (Binary)	Mnemonic
0001	LDA
0010	LDB
0011	STA
0100	STS
0110	ROR A
0111	ROR B
1000	ROL A
1001	ROL B
1010	JUMP
1011	HLT

Instruction Descriptions:

- **LDA** Loads the value from memory location into the Accumulator (A).
- **LDB 11:** Loads the value from memory location into Register B.
- **STA:** Adds the contents of Register B to the Accumulator (A) and stores the result in a memory location.
- **STS:** Subtracts the contents of Register B to the Accumulator (A) and stores the result in a memory location.
- **ROR A:** Rotates the contents of Accumulator A one bit to the right.
- **ROR B:** Rotates the contents of Accumulator B one bit to the right.
- **ROL A:** Rotates the contents of Register A one bit to the left.
- **ROL B:** Rotates the contents of Register B one bit to the left.
- **JUMP:** Transfers program control to the instruction located at memory address 2.

- **HLT:** Terminates the program execution and halts the processor.

The instruction set supports arithmetic, shifting, and control operations, allowing both data manipulation and conditional branching within a compact architecture.

6 Assembler

A tiny Python assembler converts SAP–1 assembly to a 16×8 ROM image (one byte per address 0x0–0xF). Encoding: `byte` = (`opcode` $\ll 4$) | `operand`.

Table 2: Instruction Set Summary (Opcode Map)

Mnemonic	**Opcode (Binary)**	**Hex**	**Operand**
LDA a	0001	1	yes
LDB a	0010	2	yes
STA a	0011	3	yes
STS a	0100	4	yes
ROR_A	0110	6	no
ROR_B	0111	7	no
ROL_A	1000	8	no
ROL_B	1001	9	no
JUMP a	1010	A	yes
HLT	1011	B	no

ISA map.

Syntax. `ORG a` (set PC, 0..15), `DATA x` (8-bit), labels (`LOOP:`), comments `;` or `#`. Operands accept binary (e.g., `1100`), decimal (`12`), or hex (`C`/`0xC`).

Usage. `python sap1_asm.py program.asm` → prints 16 hex bytes for the ROM (loader copies ROM→RAM).

Example: Assembly Language:

```
ORG 0
LDA 1110
LDB 1111
JUMP 0011
ORG 4
ROL_A
ROR_B
STA 1100
HLT
ORG 14
DATA 0101
DATA 1010
```

Output (0x0..0xF):

```
1E 2F A3 00 80 70 3C B0 00 00 00 00 00 00 05 0A
```

Pasted into the Logisim ROM; set `debug=1` to autoload, then `debug=0` and enable `cs_en` to run.

7 Operation

7.1 Fetch–Decode–Execute Cycle

The processor performs all instruction executions through a systematic sequence of fetch, decode, and execute phases, orchestrated by the control sequencer and the six-phase timing generator.

Fetch Phase:

- **T1:** The Program Counter (PC) outputs its current value to the bus, which is captured by the Memory Address Register (MAR).
- **T2:** The SRAM places the instruction stored at MAR on the bus (`sram_rd = 1`), and the Instruction Register (IR) latches this value. The PC increments its content by one (`pc_en = 1`), preparing for the next instruction.

Decode Phase: The opcode portion (IR[7:4]) is sent to the decoder, which asserts the corresponding instruction line (e.g., `insLDA, insADD, insSHLA`). This decoded output, when combined with the active timing pulse, determines the micro-operations required for the instruction.

Execute Phase: Depending on the instruction, data transfers and computations occur in the subsequent timing states. For instance:

- **T3:** Executes memory in and address out pin to load the decoded meaning to RAM. Common for all instructions.
- `LDA` or `LDB`: Executes memory read and register load in T4.
- `STA and STS`: Executes arithmetic addition in T4.
- `ROR`/`ROL`: Executes bit-rotating operations in T4.
- `JUMP`: Loads a new program counter value in T4.
- `HLT`: Disables timing pulses to stop the execution cycle.

The fetch–decode–execute sequence repeats continuously until a `HLT` instruction is encountered, at which point the control sequencer disables further timing signals and halts the processor.

8 Running the CPU in Automatic Mode (JUMP + ROTATE Program)

This section verifies automatic execution using a program that **loads** two bytes from memory, **jumps** to a rotate routine, performs **ROL**/**ROR** on both registers, then **stores** (`STA`, `STS`) and halts.

1) Initial Setup

[label=(h)]

1. Ensured `debug`=**LOW** (bootloader off) and `cs_en or switch`=**LOW**.
2. Turned `clk` **OFF**.
3. Pulsed `pc_reset` to set PC ← `0000`.

2) Program the ROM

1. Right-click ROM → `Edit Contents`.
2. The following 16-byte image (addresses 0x0..0xF) was entered:

```
1E 2F A3 80 90 60 70 3C 4D B0 00 00 00 00 2C 07
```

which encodes:

Addr	Bytes / Meaning
0x0..0x2	`1E` (LDA 0xE), `2F` (LDB 0xF), `A3` (JUMP 0x3)
0x3..0x6	`80` (ROL_A), `90` (ROL_B), `60` (ROR_A), `70` (ROR_B)
0x7..0x9	`3C` (STA 0xC), `4D` (STS 0xD), `B0` (HLT)
0xE..0xF	`2C`, `07` (DATA $00101100_{(44)}$, DATA $00000111_{(7)}$)

3) Load ROM → RAM (Bootloader Mode)

1. `debug`=**HIGH**. The two-phase loader copies 16 bytes to SRAM (two clocks per byte).

2. Observed MAR and Data Bus: Phase-1 latches address, Phase-2 writes data.

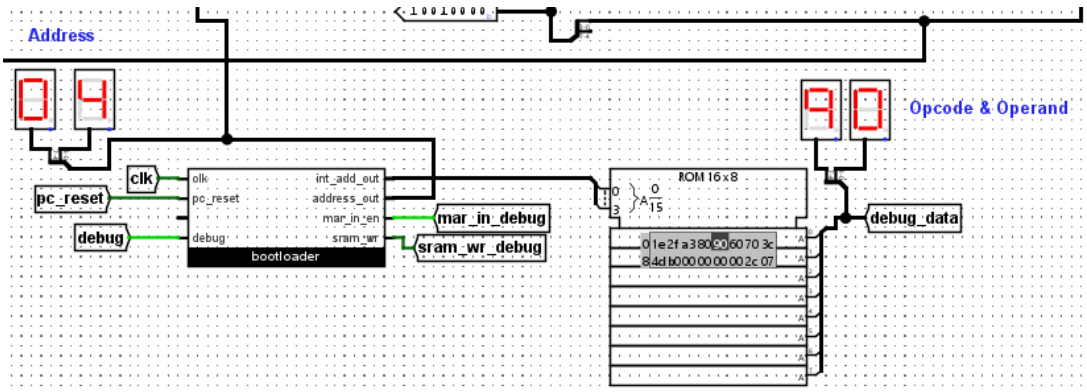

Figure 14: ROM-to-RAM transfer in `debug` mode (Phase-1: address latch, Phase-2: data write).

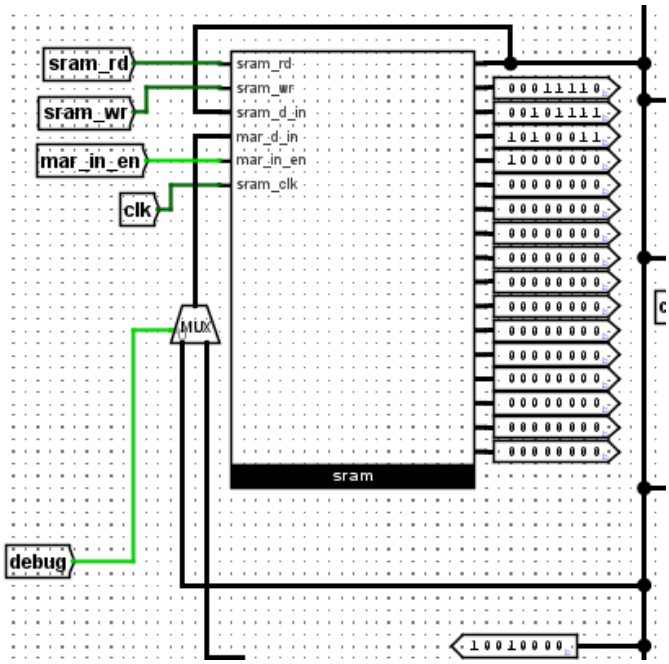

Figure 15: ROM-to-RAM transfer in `debug` mode

4) End Bootloader

1. `debug`=**LOW** (loader tri-states off).
2. Optionally give one idle clock to ensure the loader FSM is quiescent.

5) Execute in Automatic Mode

1. Pulsed `pc_reset` to `0000`.
2. `cs_en`[switch]=**HIGH**. Applied clock pulses.
3. Expected sequence:
   - **LDA 0xE**: A ← `0x2C` (00101100_2, 44).
   - **LDB 0xF**: B ← `0x07` (00000111_2, 7).
   - **JUMP 0x3**: PC ← `0x3`.
   - **ROL_A**: A rotates left once: 00101100 → 01011000 (`0x58`).
   - **ROL_B**: B rotates left once: 00000111 → 00001110 (`0x0E`).
   - **ROR_A**: A rotates right once: 01011000 → 00101100 (`0x2C`, back to original).
   - **ROR_B**: B rotates right once: 00001110 → 00000111 (`0x07`, back to original).
   - **STA 0xC**: M[`0xC`] ← (A + B) = 44 + 7 =`0x33` (51) in address 1100 0000.
   - **STS 0xD**: M[`0xD`] ← (A − B) = 44 − 7 = `0x25` (37) in address 1101 0000.
   - **HLT**: control sequencer halts.

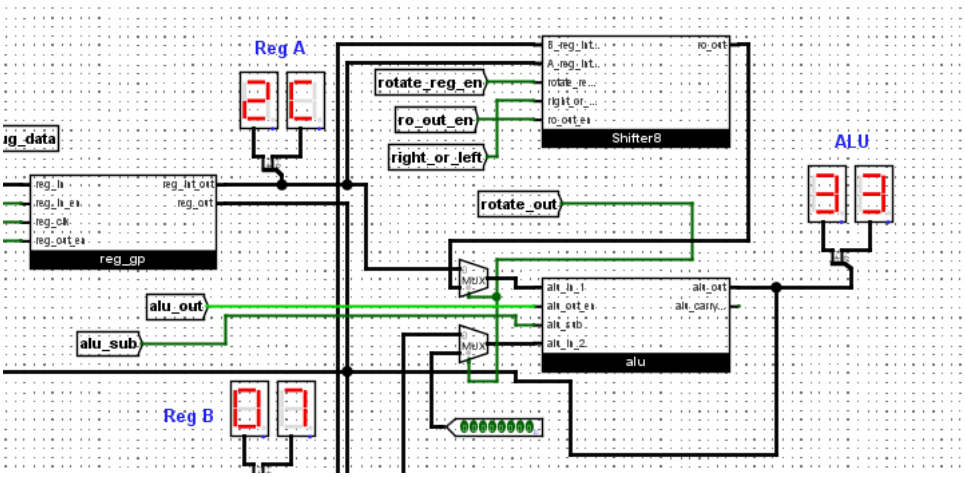

Figure 16: Values of Reg A and B and output of ALU during STA instruction

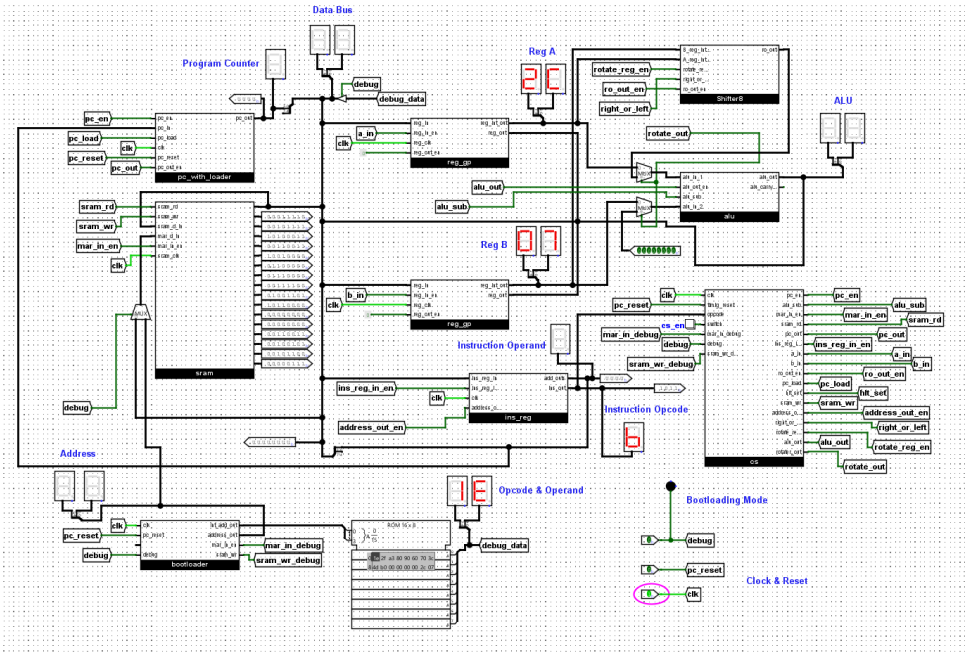

Figure 17: Final system state at `HLT`: PC ring counter idle, registers/memory hold final values.

Output Verification

- **After loads:** A=`0x2C` (44), B=`0x07` (7).
- **During ROL/ROR:** The rotated values appear on `ALU_out` only; registers A and B remain unchanged throughout the rotate sequence. Observed ALU values: ROL(A)→`0x58`, ROL(B)→`0x0E`, ROR(A)→`0x16`, ROR(B)→`0x83`.
- **Stores:** M[`0xC`]=`0x2C`; M[`0xD`]=`0x25` (since `0x2C`−`0x07`=`0x25`).
- **At HLT:** CU signals deasserted; PC/state counter idle; all displayed values hold.
- **Stores:**

$$\mathrm{M}[0xC] = \texttt{0x33}\ (51), = \ A{+}B \qquad \mathrm{M}[0xD] = \texttt{0x25}\ (37)\ =\ A{-}B.$$

- **At HLT:** All CU signals deasserted; PC/state counter stopped; displays freeze on final values.

Source (for reference):

```
ORG 0
LDA 1110
```

```
LDB 1111
JUMP 0011

ORG 3
ROL_A
ROL_B
ROR_A
ROR_B
STA 1100
STS 1101
HLT

ORG 14
DATA 00101100
DATA 00000111
ASM
```

9 Conclusion

1. The enhanced SAP-1 processor was successfully designed and implemented in Logisim Evolution, extending the classical single-bus structure with additional control and instruction functionalities.

2. The inclusion of shift and jump instructions expanded the processor's capability, enabling both arithmetic and logical data operations.

3. A six-phase ring counter and opcode-based control matrix ensured accurate timing and synchronization of all micro-operations.

4. The hardwired control sequencer demonstrated deterministic execution without microprogramming, validating the precision of timing and signal generation.

5. Dual operating modes were achieved: automatic mode for continuous instruction execution and manual/loader mode for safe program loading through handshake control.

6. Functional verification through simulation confirmed correct data transfers, instruction decoding, and bus operation across all modules.

7. The project provided a practical and educational understanding of CPU design, control logic, and micro-operation sequencing, forming a foundation for advanced processor design and experimentation.

Quick Links

GitHub: `https://github.com/Oesmita/sap1-control-sequencer_2008061/tree/main`
YouTube: https://youtu.be/<your-video-id>